# PROJECT REPORT

**Self Healing Data Pipeline Using Agentic AI**

Submitted in partial fulfillment of the requirements of the degree

**BACHELOR OF ENGINEERING**
**IN**
**INFORMATION TECHNOLOGY**

By
Alphonz George Velacherry - 03
Hriday Jain - 43
Garv Jain - 44
Aryaa Patil - 72

Guide

Dr. Anjali Malviya

Department of Information Technology

**Thadomal Shahani Engineering College**

Adv. Nari Gursahani Marg, TPS III, Off Linking Rd, Bandra West,

Mumbai, Maharashtra 400050

**University of Mumbai**

(Academic Year 2025-26)

# CERTIFICATE

This is to certify that the Mini Project entitled **"Self Healing Data Pipeline Using Agentic AI"** is a bonafide work of **Hriday Jain, Aryaa Patil, Garv Jain, Alphonz George Velacherry** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **"Bachelor of Engineering"** in **"Information Technology Engineering".**

**Dr. Anjali Malviya**

**Dr. Mukesh Israni**
Head of Department

**Prof. G. T. Thampi**
Principal

# Major Project Approval

This Mini Project entitled **"Self Healing Data Pipeline Using Agentic AI"** by **Hriday Jain, Aryaa Patil , Garv Jain, Alphonz George Velacherry** is approved for the degree of **Bachelor of Engineering** in **Information Technology Engineering.**

**Examiners**

**1……**

Dr. Anjali Malviya

**2……**

(External Examiner name & Sign)

Date:

Place: Mumbai

# TABLE OF CONTENTS


| Sr. No. | | Title | Page No. |
|---|---|---|---|
| | | Acknowledgement | i |
| | | Abstract | ii |
| **1** | | **Introduction** | 1 |
| | 1.1 | Background of the Study | 1 |
| | 1.2 | Problem Statement | 2 |
| | 1.3 | Objectives | 2 |
| | 1.4 | Scope of the Project | 3 |
| | 1.5 | Significance of the Project | 3 |
| **2** | | **Literature Survey** | 5 |
| | 2.1 | Introduction | 5 |
| | 2.2 | Traditional ETL and ELT Architectures | 5 |
| | 2.3 | Data Lakes, Warehouses and Modern Architectures | 6 |
| | 2.4 | Data Observability and Monitoring Systems | 6 |
| | 2.5 | Autonomic Computing and Self-Healing Systems | 7 |
| | 2.6 | Machine Learning for Anomaly Detection | 7 |
| | 2.7 | Multi-Agent Systems and Distributed Intelligence | 7 |
| | 2.8 | Large Language Models in System Diagnosis | 8 |
| | 2.9 | Streaming Systems and Real-Time Pipelines | 8 |
| | 2.10 | MLOps and Pipeline Automation | 8 |
| | 2.11 | Reinforcement Learning and Adaptive Systems | 9 |
| | 2.12 | Integrated AI-Driven Pipeline Architectures | 9 |
| | 2.13 | Research Gaps and Motivation | 9 |
| | 2.14 | Summary | 10 |
| **3** | | **System Requirements and Analysis** | 11 |
| | 3.1 | Introduction | 11 |

3.2 System Overview 11
3.3 Functional Requirements 12
3.4 Non-Functional Requirements 14
3.5 System Actors and Use Cases 15
3.6 Feasibility Study 16
3.7 Risk Analysis and Mitigation 17
3.8 Summary 17
**4 System Design** 18
4.1 Introduction 18
4.2 Design Objectives 18
4.3 System Architecture 18
4.4 Agent-Based Architecture 20
4.5 Data Flow Design 21
4.6 Failure Injection Module 21
4.7 Similarity-Based Resolution Module 22
4.8 LLM Integration 22
4.9 Data Storage Design 22
4.10 Backend Design 23
4.11 Frontend Design 23
4.12 System Workflow Summary 23
4.13 Summary 24
**5 Implementation** 25
5.1 Introduction 25
5.2 Overall Implementation Approach 25
5.3 Project Structure 25
5.4 Backend Implementation 26
5.5 Failure Injection Module 27
5.6 Anomaly Detection Implementation 27
5.7 Similarity-Based Resolution 28

5.8 LLM-Based Remediation 28
5.9 Agentic Workflow Implementation 29
5.10 Data Storage and Management 29
5.11 Frontend Implementation 29
5.12 System Execution Flow 30
5.13 Summary 30
**6 Results** 31
6.1 Introduction 31
6.2 Processing Time Analysis 31
6.3 Resolution Method Distribution 32
6.4 Error Type Timing Analysis 32
6.5 Error Combination Analysis 33
6.6 Error Type Resolution Success 33
6.7 Comparison with Traditional Pipelines 34
6.8 Summary 34
**7 Advantages and Limitations** 35
7.1 Advantages 35
7.2 Limitations 35
**8 Future Scope** 37
**9 Conclusion** 38
**References** 39

# ACKNOWLEDGEMENT

This project has consumed a significant amount of time and resources, and we would like to thank everyone who has contributed to its success. We would like to express our deepest gratitude to the project head, **Dr. Anjali Malviya** for her time, patience and direction, as well as for inspiring us throughout. Without their guidance and support, this project would not have been possible. Her expertise, insights, and encouragement were invaluable in helping us navigate the various challenges and complexities of this project.

We would also like to extend our sincere thanks to the authors of the many research papers that we consulted throughout the course of this project. The insights and ideas presented in these papers were instrumental in helping us develop a deeper understanding of the subject matter and in shaping the direction of our research.

We are also grateful to our colleagues and friends who provided us with valuable feedback and support throughout the project. Their encouragement and insights were invaluable in helping us refine our ideas.

Finally, we would like to thank the Department of Information Technology at Thadomal Shahani Engineering College for providing us with the necessary resources and environment to complete this project successfully.

Our team consisted of the following members:
Alphonz George Velacherry - 03
Garv Jain - 42
Hriday Jain - 43
Aarya Patil - 72

# ABSTRACT

Self-Healing Data Pipelines Using Agentic AI presents a simulation-driven framework for building autonomous, resilient data pipelines capable of detecting, diagnosing, and repairing data quality and operational failures with minimal human intervention. Modern data engineering systems rely heavily on Extract–Transform–Load (ETL) pipelines, which are inherently fragile due to evolving data schemas, inconsistent input formats, missing values, and transformation errors. Traditional approaches to pipeline maintenance are largely reactive, requiring significant manual monitoring and debugging effort, leading to increased downtime, operational costs, and reduced data reliability.

This project addresses these limitations by introducing an agent-based architecture that models a self-healing pipeline using a controlled experimental setup. A clean reference dataset stored in a SQLite database is used as the baseline, from which data batches are periodically sampled and deliberately corrupted using a failure injection module. These injected failures simulate real-world issues such as schema drift, null values in key fields, duplicate records, malformed timestamps, and inconsistent data types, enabling systematic evaluation of detection and recovery mechanisms.

The proposed system employs a multi-agent workflow consisting of monitoring, diagnosis, decision, execution, and audit agents. The monitoring agent continuously evaluates incoming data batches and extracts telemetry features, while the diagnosis agent classifies anomalies using machine learning models such as Random Forest and One-Class Support Vector Machines. Based on the detected issue, the decision agent selects an appropriate remediation strategy from three pathways: rule-based fixes, similarity-based retrieval using a TF-IDF and cosine similarity module, or generative reasoning using Large Language Models (LLMs) for complex or previously unseen failures.

The execution agent applies corrective actions to produce a healed dataset, while the audit agent maintains detailed logs, system metrics, and resolution histories to support traceability and continuous improvement. A human-in-the-loop mechanism is incorporated to review and approve high-risk remediation actions, ensuring safety and preventing unintended data corruption. Additionally, the system provides a web-based monitoring dashboard for real-time visualization of pipeline health, detected anomalies, and resolution outcomes.

Experimental evaluation demonstrates that the proposed framework effectively reduces Mean Time to Detection (MTTD) and Mean Time to Resolution (MTTR) in simulated environments, highlighting the potential of integrating agentic AI and generative reasoning into data engineering workflows. The project establishes a practical foundation for future research in autonomous data systems and contributes toward the development of intelligent, adaptive, and self-healing data infrastructures.

# CHAPTER 1 - INTRODUCTION

## 1.1 Background of the Study

In the era of data-driven decision-making, organizations increasingly rely on data pipelines to ingest, process, and deliver data for analytics, machine learning, and business intelligence. These pipelines, commonly implemented using Extract–Transform–Load (ETL) or Extract–Load–Transform (ELT) paradigms, form the backbone of modern data engineering systems. However, as data sources become more diverse and dynamic, pipelines are exposed to a wide range of challenges, including schema drift, inconsistent data formats, missing or corrupted records, and unexpected system failures.

Traditional data pipelines are typically designed as static, rule-based systems that depend heavily on predefined schemas and transformation logic. While effective in controlled environments, these systems lack adaptability when faced with evolving data conditions. As a result, failures often propagate through the pipeline, leading to incorrect outputs, delayed processing, and unreliable analytics. Industry studies indicate that data engineering teams spend a significant portion of their time identifying and resolving recurring pipeline issues, highlighting the need for more intelligent and autonomous solutions.

Recent advancements in artificial intelligence and distributed systems have introduced the concept of self-healing systems, which are capable of automatically detecting anomalies, diagnosing root causes, and applying corrective actions without manual intervention. This paradigm shift is closely related to autonomic computing, which emphasizes self-monitoring, self-optimization, and self-repair. In the context of data engineering, these ideas have led to the emergence of self-healing data pipelines that integrate machine learning, agent-based architectures, and generative AI techniques to achieve resilience and adaptability.

This project builds upon these developments by proposing a self-healing data pipeline framework using Agentic AI. The system models a multi-agent workflow in which specialized agents collaboratively monitor data flow, detect anomalies, determine appropriate remediation strategies, and execute corrective actions. By combining classical machine learning techniques with modern generative AI capabilities, the system aims to transform traditional pipelines from reactive systems into adaptive, semi-autonomous infrastructures.

## 1.2 Problem Statement

Despite their critical role in modern data ecosystems, traditional data pipelines suffer from several limitations that hinder efficiency and reliability. One of the primary issues is the reactive nature of failure handling. Most systems rely on alerts and manual debugging, which increases Mean Time to Detection (MTTD) and Mean Time to Resolution (MTTR). This delay can significantly impact downstream applications, especially in time-sensitive analytics.

Another major challenge is the lack of adaptability. Data pipelines are often tightly coupled with predefined schemas and transformation rules, making them vulnerable to changes in upstream data sources. Even minor schema modifications can cause pipeline failures, requiring manual intervention to restore functionality.

Additionally, silent data failures pose a critical risk. These occur when data appears structurally valid but contains semantic inconsistencies or inaccuracies, leading to incorrect analytical outcomes without immediate detection. Existing validation mechanisms primarily focus on syntactic correctness and fail to capture deeper contextual errors.

Furthermore, the increasing scale and complexity of data systems make it impractical to rely solely on human intervention for monitoring and maintenance. This creates a need for intelligent systems that can autonomously manage pipeline health while ensuring safety and reliability.

## 1.3 Objectives

The primary objective of this project is to design and implement a self-healing data pipeline framework using an Agentic AI approach. The system aims to reduce dependency on manual intervention by enabling autonomous detection, diagnosis, and resolution of data pipeline issues.

The specific objectives of the project are as follows:

- To design a simulation-based data pipeline environment capable of replicating real-world data quality issues through controlled failure injection.
- To develop a multi-agent architecture consisting of monitoring, diagnosis, decision, execution, and audit agents for modular and scalable pipeline management.
- To implement anomaly detection using machine learning techniques such as Random Forest and One-Class Support Vector Machines.
- To incorporate a similarity-based resolution mechanism using TF-IDF and cosine similarity for retrieving past solutions.

- To integrate Large Language Models (LLMs) for generating context-aware remediation strategies for complex or previously unseen failures.
- To implement a human-in-the-loop approval mechanism for high-risk actions to ensure system safety and data integrity.
- To develop a web-based dashboard for real-time monitoring, visualization, and control of the pipeline.
- To evaluate system performance using metrics such as detection accuracy, resolution success rate, MTTD, and MTTR.

## 1.4 Scope of the Project

The scope of this project is limited to the design and implementation of a self-healing data pipeline within a controlled simulation environment. The system focuses on demonstrating the feasibility of integrating agent-based orchestration, machine learning, and generative AI for autonomous pipeline management.

The implementation is carried out using a local setup with a SQLite database serving as the source of clean data. Failure scenarios are artificially generated using a failure injection module, allowing systematic testing of detection and remediation strategies. The system processes data in batch cycles rather than real-time streaming environments.

The project does not aim to replace production-grade data pipeline tools but rather to provide a conceptual and practical framework that can be extended to real-world systems. Advanced features such as distributed processing, large-scale deployment, and real-time streaming integration are considered outside the current scope and are identified as areas for future work.

## 1.5 Significance of the Project

The proposed system represents an important step toward the development of intelligent and autonomous data engineering solutions. By reducing reliance on manual monitoring and intervention, self-healing pipelines can significantly improve operational efficiency and reliability.

The integration of agent-based architectures enables modular and scalable system design, where each agent performs a specific function while collaborating with others to achieve overall system goals. The use of machine learning enhances the system's ability to detect anomalies accurately, while generative AI introduces flexibility in handling complex and unforeseen scenarios.

The inclusion of a human-in-the-loop mechanism ensures that automation does not compromise safety, providing a balance between autonomy and control. This is particularly important in data-critical environments where incorrect actions can have significant consequences.

Overall, this project contributes to the growing field of intelligent data systems by demonstrating how Agentic AI can be applied to build adaptive, resilient, and self-improving data pipelines. It provides a foundation for future research and development in autonomous data engineering and highlights the potential for integrating AI-driven solutions into real-world data workflows.

# CHAPTER 2 - LITERATURE SURVEY

## 2.1 Introduction

The rapid growth of data-driven systems has significantly increased the complexity of data pipelines, necessitating more intelligent and resilient architectures. Traditional approaches to data engineering are increasingly inadequate in handling modern challenges such as high data velocity, schema evolution, heterogeneous data sources, and real-time processing requirements. As a result, research has evolved from static ETL systems toward adaptive, autonomous, and self-healing data pipelines.

This chapter provides a comprehensive review of foundational and contemporary research across multiple domains relevant to this project, including traditional ETL architectures, data observability, autonomic computing, machine learning–based anomaly detection, multi-agent systems, Large Language Models (LLMs), MLOps practices, and streaming data systems. The objective is to establish the theoretical and technological foundations that motivate the proposed self-healing pipeline framework.

## 2.2 Traditional ETL and ELT Architectures

Extract–Transform–Load (ETL) systems have long served as the backbone of enterprise data integration. These systems consolidate data from heterogeneous sources into centralized repositories for analytics and reporting. With the rise of cloud computing, the Extract–Load–Transform (ELT) paradigm has gained popularity, where transformations are applied after loading data into scalable storage systems.

Despite their widespread adoption, traditional ETL/ELT systems suffer from several structural limitations. These pipelines are typically designed using static workflows with tightly coupled components, making them brittle and difficult to adapt to changing data schemas or formats. Even minor inconsistencies in input data can lead to pipeline failures, requiring manual intervention.

Research by Patel and Arora [2] indicates that data engineers spend a substantial portion of their time maintaining pipeline reliability rather than developing new data capabilities. Furthermore, tightly coupled architectures limit scalability, as entire pipelines must be scaled even when only specific components require additional resources [17].

These limitations have driven the need for more flexible and adaptive pipeline architectures capable of handling dynamic data environments.

## 2.3 Data Lakes, Warehouses, and Modern Data Architectures

Modern data architectures have evolved beyond traditional data warehouses to include data lakes and lakehouse architectures. Data lakes allow storage of raw, unstructured, and semi-structured data, enabling greater flexibility in data processing. Technologies such as Apache Hadoop [23] and Apache Spark [22] have enabled distributed processing of large-scale datasets.

However, while these systems improve scalability and storage flexibility, they introduce new challenges related to data quality and governance. Without strict schema enforcement, data lakes often become "data swamps," where inconsistent and unreliable data accumulates over time.

Lakehouse architectures attempt to address these issues by combining the reliability of data warehouses with the flexibility of data lakes. Nevertheless, the problem of maintaining data quality across complex pipelines remains largely unresolved, reinforcing the need for automated monitoring and self-healing mechanisms.

## 2.4 Data Observability and Monitoring Systems

Data observability has emerged as a critical discipline for ensuring pipeline reliability. Observability systems monitor various dimensions of data health, including freshness, schema consistency, distribution changes, and volume anomalies. Industry tools such as Monte Carlo and Datafold provide automated monitoring and alerting capabilities [3], [4].

While these systems improve visibility into pipeline operations, they remain fundamentally reactive. They can detect anomalies but do not provide mechanisms for autonomous remediation. Engineers must still manually investigate alerts, identify root causes, and implement fixes.

Additionally, observability systems often rely on statistical thresholds and heuristics, which may fail to capture semantic inconsistencies or context-specific errors. Cormack [16] highlights the issue of silent data failures, where incorrect data passes validation checks but leads to erroneous analytical outcomes.

This gap between detection and resolution motivates the development of self-healing systems that go beyond observability to include automated diagnosis and correction.

## 2.5 Autonomic Computing and Self-Healing Systems

The concept of autonomic computing, introduced by Kephart and Chess [1], provides the theoretical foundation for self-healing systems. The MAPE-K loop (Monitor, Analyze, Plan, Execute, Knowledge) defines a feedback-driven approach to system self-management.

Self-healing systems extend this concept by enabling automatic detection and correction of failures. In recent years, this paradigm has been applied to machine learning systems through frameworks such as Self-Healing Machine Learning (SHML) [5], which allow models to adapt dynamically to changing data distributions.

In the context of data pipelines, self-healing systems aim to reduce human intervention by automating the entire lifecycle of anomaly detection, root cause analysis, and remediation. However, most existing implementations are limited in scope and do not fully integrate multiple AI techniques into a unified framework.

## 2.6 Machine Learning for Anomaly Detection

Machine learning plays a central role in enabling intelligent data pipelines. Supervised models such as Random Forest and Support Vector Machines are effective for detecting known anomalies, while unsupervised methods such as Isolation Forest are used for identifying unknown or rare patterns [6].

Time-series forecasting models further enhance detection capabilities by predicting expected system behavior and identifying deviations in real time [7]. These approaches enable proactive monitoring and reduce detection latency.

However, machine learning models primarily focus on detection rather than remediation. They can identify anomalies but do not provide actionable solutions. This limitation necessitates the integration of decision-making and execution mechanisms to achieve true autonomy.

## 2.7 Multi-Agent Systems and Distributed Intelligence

Multi-agent systems provide a scalable framework for implementing distributed intelligence. Agents are autonomous entities capable of perception, reasoning, and action [8]. In complex systems, multiple agents collaborate to achieve shared objectives.

Recent research has explored the application of agent-based architectures in data systems, where agents perform specialized roles such as monitoring, anomaly detection, and optimization [9]. This modular approach improves scalability and fault tolerance.

Agentic AI extends traditional multi-agent systems by incorporating learning and reasoning capabilities, enabling agents to adapt dynamically to changing environments. In the context of data pipelines, agent-based architectures facilitate decentralized decision-making and enable flexible orchestration of complex workflows.

## 2.8 Large Language Models in System Diagnosis

Large Language Models (LLMs) have introduced new possibilities for automated reasoning and diagnosis. These models are capable of understanding unstructured data such as logs and error messages, making them valuable for root cause analysis.

Research such as LogParser-LLM [10] demonstrates the effectiveness of LLMs in parsing and clustering system logs, while LLMParser [11] highlights their ability to generalize across unseen patterns. These capabilities enable LLMs to generate contextual explanations and suggest remediation strategies.

However, LLMs are not inherently reliable and may produce incorrect or misleading outputs. This necessitates the inclusion of validation mechanisms and human oversight when integrating LLMs into critical systems.

## 2.9 Streaming Systems and Real-Time Data Pipelines

Modern data pipelines increasingly operate in real-time environments, processing streaming data from sources such as sensors, applications, and user interactions. Technologies such as Apache Kafka [25] enable distributed messaging and real-time data processing.

While streaming systems improve responsiveness and scalability, they introduce additional challenges related to fault tolerance and data consistency. Failures in streaming pipelines can propagate rapidly, making timely detection and remediation critical.

Self-healing mechanisms are particularly valuable in streaming environments, where delays in failure resolution can have immediate downstream impacts.

## 2.10 MLOps and Pipeline Automation

MLOps practices focus on automating the lifecycle of machine learning systems, including model training, deployment, monitoring, and maintenance. These practices emphasize reproducibility, scalability, and continuous integration.

While MLOps provides frameworks for managing machine learning workflows, it does not fully address data pipeline failures. Integrating MLOps principles with data engineering practices can enhance pipeline reliability and enable continuous improvement.

## 2.11 Reinforcement Learning and Adaptive Systems

Reinforcement learning (RL) has been applied to optimize system behavior through trial-and-error learning. In data pipelines, RL can be used to optimize scheduling, resource allocation, and remediation strategies [14].

RL-based systems can adapt over time by learning from past actions and outcomes. However, they require extensive training data and careful design to ensure stability and reliability.

## 2.12 Integrated AI-Driven Pipeline Architectures

Recent research emphasizes the need for integrating multiple AI techniques into unified frameworks. Combining machine learning for detection, agent-based systems for orchestration, and LLMs for reasoning can enable fully autonomous pipelines.

AI-based data cleansing techniques [15] and adaptive orchestration strategies [13] demonstrate the potential of such integration. However, most existing systems focus on individual components rather than holistic solutions.

## 2.13 Research Gaps and Motivation

Despite significant advancements, several challenges remain:

- Lack of unified frameworks integrating detection, diagnosis, and remediation
- Limited ability to handle unseen or complex failures
- Absence of standardized evaluation benchmarks
- Challenges in ensuring safe and reliable automation
- Need for effective human-AI collaboration

The proposed system addresses these gaps by combining machine learning, agent-based orchestration, and LLM-driven reasoning into a cohesive self-healing pipeline framework.

## 2.14 Summary

This chapter reviewed the evolution of data pipeline technologies and the role of artificial intelligence in enhancing their resilience and adaptability. Traditional ETL systems, while foundational, are insufficient for modern data environments. Emerging approaches such as data observability, autonomic computing, machine learning, multi-agent systems, and LLMs provide the building blocks for self-healing pipelines.

The integration of these technologies presents a promising direction for developing intelligent, autonomous data systems. The next chapter presents the system requirements and analysis for the proposed framework.

# CHAPTER 3 - SYSTEM REQUIREMENT AND ANALYSIS

## 3.1 Introduction

This chapter presents the system requirements and analysis for the proposed Self-Healing Data Pipeline using Agentic AI. The system is implemented as a controlled simulation of a data pipeline, designed to demonstrate how intelligent agents can detect, diagnose, and resolve data quality issues with minimal human intervention.

Unlike production-grade pipelines, the system operates in periodic batch cycles, where clean data is sampled, intentionally corrupted, and processed through an agent-based workflow. This approach allows systematic evaluation of anomaly detection and remediation strategies in a reproducible environment.

The chapter outlines the functional and non-functional requirements, system actors, workflows, feasibility, and associated risks, ensuring that the design is both practical and aligned with the implemented system.

## 3.2 System Overview

The proposed system is a simulation-driven data pipeline that mimics real-world data processing scenarios. It uses a clean reference dataset stored in a SQLite database, from which batches are sampled during each execution cycle.

A failure injection module deliberately introduces data quality issues such as missing values, schema inconsistencies, duplicate records, and malformed data fields. These corrupted batches are then processed by a multi-agent system designed to simulate a self-healing pipeline.

The system operates in iterative cycles rather than continuous real-time processing. Each cycle consists of data ingestion, validation, anomaly detection, decision-making, remediation, and logging.

The architecture is composed of the following agents:

- Monitoring Agent
- Diagnosis Agent
- Decision Agent

- Execution Agent
- Audit Agent

These agents collaboratively perform the tasks required for semi-autonomous pipeline operation.

## 3.3 Functional Requirements

Functional requirements define the core operations of the system based on the actual implementation.

### 3.3.1 Batch-Based Monitoring

The system monitors data in periodic simulation cycles rather than continuous streaming. During each cycle, the monitoring component evaluates:

- Dataset structure
- Record counts
- Presence of null values
- Data type consistency
- Schema integrity

This information is used to identify potential anomalies in the dataset.

### 3.3.2 Anomaly Detection and Classification

The system detects anomalies using a combination of rule-based validation and lightweight machine learning models.

The types of anomalies include:

- Missing or null values
- Schema mismatches
- Duplicate records
- Invalid data formats
- Orphan relational records

Machine learning models such as Random Forest and One-Class Support Vector Machines are used as

supporting tools for classification and outlier detection. These models operate on extracted features rather than large-scale training pipelines.

### 3.3.3 Semi-Autonomous Remediation

The system performs remediation using a hybrid approach:

1. **Rule-Based Fixes**
   Predefined corrections for common issues (e.g., filling null values, removing duplicates)
2. **Similarity-Based Retrieval**
   A TF-IDF and cosine similarity module retrieves previously applied solutions for similar issues
3. **LLM-Based Suggestions**
   For complex or unknown issues, a Large Language Model generates remediation suggestions

The system is semi-autonomous, meaning that:

- Low-risk fixes are applied automatically
- High-risk fixes require human approval

### 3.3.4 Agent-Based Workflow

The system implements an agent-based architecture where each agent performs a specific role:

- Monitoring Agent: Identifies anomalies in each batch
- Diagnosis Agent: Classifies the type and cause of issues
- Decision Agent: Selects the appropriate resolution strategy
- Execution Agent: Applies the selected fix
- Audit Agent: Records all actions, logs, and outcomes

Agents communicate through structured data exchange and operate sequentially within each cycle.

### 3.3.5 Human-in-the-Loop Approval

The system includes a human approval mechanism for high-risk operations. Actions such as structural changes or uncertain fixes are placed in an approval queue.

This ensures:

- Data safety
- Controlled automation
- Prevention of unintended modifications

### 3.3.6 Monitoring Dashboard

A web-based interface provides visibility into system operations, including:

- Pipeline status
- Detected anomalies
- Applied fixes
- Batch history and logs

The dashboard retrieves data through backend API endpoints and updates periodically.

## 3.4 Non-Functional Requirements

### 3.4.1 Performance

- The system should process each batch within a reasonable time frame
- Detection and remediation should occur within the same cycle

### 3.4.2 Reliability

- The system should consistently detect known anomalies
- All operations must be logged for traceability

### 3.4.3 Accuracy

- Detection mechanisms should minimize false positives and false negatives
- Resolution strategies should maintain data integrity

### 3.4.4 Safety

- Automated actions must be restricted to safe operations
- Risky actions must require human approval

### 3.4.5 Maintainability

- The system should be modular and easy to extend
- Individual components (agents) should be independently testable

### 3.4.6 Scalability (Conceptual)

The current implementation is local and simulation-based. However, the modular architecture is designed such that it can be extended to larger datasets and distributed environments in future work.

## 3.5 System Actors and Use Cases

### 3.5.1 Actors

- System Agents: Perform automated processing tasks
- User (Operator): Monitors system and approves actions

### 3.5.2 Use Cases

**UC1: System Initialization**

- Backend server starts
- Agents are initialized

**UC2: Batch Generation and Failure Injection**

- Clean data is sampled
- Errors are injected

**UC3: Anomaly Detection**

- Monitoring agent evaluates batch
- Issues are flagged

**UC4: Diagnosis and Decision**

- Root cause identified
- Resolution strategy selected

**UC5: Execution of Fixes**

- Fixes applied to dataset
- Cleaned output generated

**UC6: Audit and Logging**

- Events recorded
- Reports generated

**UC7: Human Approval (if required)**

- High-risk actions reviewed
- User approves or rejects

## 3.6 Feasibility Study

### 3.6.1 Technical Feasibility

The system is implemented using widely available tools:

- Python for backend logic
- Scikit-learn for ML models
- SQLite for data storage
- HTML/CSS/JS for frontend

The architecture is feasible within an academic environment.

### 3.6.2 Economic Feasibility

The system uses open-source tools and minimal resources. It can run on standard hardware, making it cost-effective for development and testing.

### 3.6.3 Operational Feasibility

The system is designed for demonstration and experimentation. It can be operated by users with basic knowledge of data processing and machine learning.

## 3.7 Risk Analysis and Mitigation

| Risk | Impact | Mitigation |
|---|---|---|
| Incorrect anomaly detection | Wrong fixes applied | Combine rule-based and ML detection |
| LLM incorrect suggestions | Invalid remediation | Human approval required |
| Data corruption | Loss of integrity | Restrict unsafe operations |
| Performance delays | Slower cycles | Optimize batch size |
| System errors | Pipeline interruption | Logging and restart capability |

Risk and Impact

## 3.8 Summary

This chapter defined the system requirements and provided a realistic analysis of the proposed self-healing data pipeline. The design reflects a semi-autonomous, batch-based system that integrates agent-based workflows, machine learning detection, and LLM-assisted reasoning.

The next chapter presents the system design, including architecture, data flow, and component interactions, based on these requirements.

# CHAPTER 4 - IMPLEMENTATION

## 4.1 Introduction

This chapter presents the design of the Self-Healing Data Pipeline using Agentic AI. The system is implemented as a simulation-based pipeline that models real-world data processing and failure recovery using an agent-based architecture. The design focuses on modularity, traceability, and controlled automation, enabling the system to detect, diagnose, and resolve data anomalies within batch processing cycles.

The system is structured into distinct layers and components to ensure clarity, maintainability, and extensibility. This chapter describes the overall architecture, agent interactions, data flow, storage mechanisms, and technology stack used in the implementation.

## 4.2 Design Objectives

The key objectives guiding the system design are:

- To create a modular pipeline that separates data processing and decision-making logic
- To implement an agent-based architecture for structured task delegation
- To simulate real-world failures using a controlled failure injection mechanism
- To enable semi-autonomous remediation with safety constraints
- To ensure complete traceability through logging and audit mechanisms
- To provide a user interface for monitoring and control

## 4.3 System Architecture

The system follows a three-layer architecture, consisting of:

1. Presentation Layer
2. Processing Layer
3. Intelligence Layer (Agentic Layer)

### 4.3.1 Presentation Layer

The presentation layer provides the user interface for interacting with the system. It is implemented using HTML, CSS, and JavaScript.

Functions include:

- Uploading datasets
- Viewing pipeline status
- Monitoring anomalies and fixes
- Accessing logs and reports
- Approving or rejecting high-risk actions

The UI communicates with the backend through REST API endpoints and updates periodically.

### 4.3.2 Processing Layer

The processing layer handles all deterministic data operations. It is responsible for:

- Data ingestion from local storage
- Failure injection (simulated errors)
- Data validation and preprocessing
- Application of corrective transformations
- Storage of processed and healed data

This layer ensures that all data operations are repeatable and controlled.

### 4.3.3 Intelligence Layer (Agentic Layer)

The intelligence layer is the core of the system and implements the agent-based workflow. It is responsible for:

- Detecting anomalies
- Diagnosing issues

- Selecting remediation strategies
- Executing corrective actions
- Maintaining system logs

This layer enables semi-autonomous behavior through coordinated agent interactions.

## 4.4 Agent-Based Architecture

The system uses a multi-agent design where each agent has a clearly defined role.

### 4.4.1 Monitoring Agent

- Scans each batch for anomalies
- Extracts features such as null counts, schema differences, and record inconsistencies
- Flags potential issues

### 4.4.2 Diagnosis Agent

- Identifies the type of anomaly
- Determines affected fields and records
- Categorizes errors (e.g., missing values, schema mismatch, invalid format)

### 4.4.3 Decision Agent

- Selects the appropriate remediation strategy
- Chooses between:
    - Rule-based fixes
    - Similarity-based retrieval
    - LLM-generated suggestions
- Evaluates confidence levels

### 4.4.4 Execution Agent

- Applies the selected fix to the dataset

- Updates the processed data
- Ensures consistency after correction

### 4.4.5 Audit Agent

- Maintains logs of all actions
- Stores batch history and metrics
- Generates reports for analysis and debugging

## 4.5 Data Flow Design

The system operates in sequential batch cycles. The data flow is as follows:

1. **Data Sampling**
   Clean data is retrieved from the SQLite database
2. **Failure Injection**
   Artificial errors are introduced into the dataset
3. **Monitoring Phase**
   Data is analyzed for anomalies
4. **Diagnosis Phase**
   Issues are classified and localized
5. **Decision Phase**
   Appropriate resolution strategy is selected
6. **Execution Phase**
   Fixes are applied to generate a cleaned dataset
7. **Audit Phase**
   Results and logs are stored

This pipeline is repeated for each cycle, enabling continuous evaluation and improvement.

## 4.6 Failure Injection Module

The failure injection module simulates real-world data issues. It introduces controlled errors such as:

- Missing columns
- Null primary keys
- Duplicate records
- Invalid data types
- Malformed timestamps
- Orphan relational entries

This module is essential for testing the robustness of the self-healing mechanism.

## 4.7 Similarity-Based Resolution Module

The system includes a TF-IDF–based similarity module that:

- Stores past issues and their resolutions
- Converts issue descriptions into vector representations
- Uses cosine similarity to find the closest match

This enables reuse of previous solutions and reduces dependency on external reasoning systems.

## 4.8 LLM Integration

For complex or unknown issues, the system uses a Large Language Model to:

- Generate diagnostic explanations
- Suggest remediation strategies
- Provide human-readable summaries

LLM outputs are treated as suggestions and are subject to validation and human approval when necessary.

## 4.9 Data Storage Design

The system uses a structured local storage approach:

- SQLite Database: Stores clean reference data

- CSV Files: Store processed and healed datasets
- Logs Directory: Stores system logs and reports
- Model Storage: Stores trained ML models

This structure ensures easy access, traceability, and reproducibility.

## 4.10 Backend Design

The backend is implemented using a Python HTTP server. It provides API endpoints such as:

- `/api/state` → System status
- `/api/action` → Control pipeline execution
- `/api/batch/{id}` → Batch-specific data
- `/api/report` → Generate reports

The backend handles data processing, agent coordination, and communication with the frontend.

## 4.11 Frontend Design

The frontend provides a control dashboard with features such as:

- Real-time pipeline monitoring
- Visualization of anomalies and fixes
- Batch history tracking
- Approval queue interface
- Report generation

The interface updates periodically by polling backend APIs.

## 4.12 System Workflow Summary

The complete workflow integrates all components:

- Data enters through ingestion

- Failures are injected
- Agents process data sequentially
- Fixes are applied
- Results are logged and displayed

This workflow demonstrates how agent-based systems can simulate self-healing behavior in data pipelines.

## 4.13 Summary

This chapter described the system design for the proposed self-healing data pipeline. The architecture combines a layered design with an agent-based workflow to enable modular and controlled automation. Each component plays a specific role in ensuring that data anomalies are detected, analyzed, and resolved effectively.

The next chapter presents the implementation details, including code structure, backend logic, and system components.

# CHAPTER 5 - SYSTEM DESIGN

## 5.1 Introduction

This chapter describes the implementation of the Self-Healing Data Pipeline using Agentic AI. The system is developed as a simulation-based pipeline that models real-world data processing and anomaly handling.

The system is built primarily using Python for backend processing, with a lightweight web interface for monitoring and interaction. It integrates rule-based logic, machine learning models, and Large Language Model (LLM)–based reasoning to achieve semi-autonomous data pipeline behavior.

## 5.2 Overall Implementation Approach

The system operates in iterative batch cycles. During each cycle:

1. Clean data is sampled from a SQLite database
2. Artificial errors are introduced using a failure injection module
3. The corrupted dataset is processed by a sequence of agents
4. Anomalies are detected, diagnosed, and resolved
5. Results are logged and displayed through the dashboard

This approach allows controlled testing of the self-healing mechanism under different failure scenarios.

## 5.3 Project Structure

The project is organized into modular components to ensure clarity and maintainability. The key files and directories include:

- **server.py**
  Acts as the main backend server. It handles API requests, serves frontend files, and manages the pipeline execution.
- **pipeline_utils.py**
  Contains utility functions for data loading, validation, anomaly detection, and transformation.

- **generated_healers.py**
  Stores reusable healing functions, including both predefined fixes and dynamically generated solutions.

- **scripts/**
  Contains data preparation scripts, including dataset preprocessing and setup.

- **web/**
  Includes frontend files such as HTML, CSS, and JavaScript for the monitoring dashboard.

- **data/**
  Stores raw, processed, and cleaned datasets, along with logs and reports.

## 5.4 Backend Implementation

### 5.4.1 Server Architecture

The backend is implemented using Python's HTTP server with support for concurrent request handling. It exposes REST API endpoints that enable interaction between the frontend and the processing pipeline.

Key functionalities include:

- Handling pipeline execution commands
- Serving system state and metrics
- Providing access to batch data and logs
- Generating analysis reports

### 5.4.2 API Endpoints

The system exposes several API endpoints:

- `/api/state`
  Returns the current system status, including metrics, batch information, and anomaly summaries

- `/api/action`
  Controls pipeline execution (start, stop, run cycle)

- `/api/batch/{id}`
  Provides detailed information about a specific batch

- `/api/healed-data`

  Returns processed datasets after remediation

- `/api/report`

  Generates downloadable reports

These endpoints enable seamless communication between frontend and backend.

## 5.5 Failure Injection Module

The failure injection module is a key component of the system. It introduces controlled errors into clean datasets to simulate real-world data issues.

Types of injected failures include:

- Missing columns
- Null values in key fields
- Duplicate records
- Invalid data types
- Malformed timestamps
- Orphan relational records

This module ensures that the system is tested against diverse and realistic anomaly scenarios.

## 5.6 Anomaly Detection Implementation

Anomaly detection is implemented using a hybrid approach:

### 5.6.1 Rule-Based Detection

Basic validation rules are applied to detect:

- Missing values
- Schema inconsistencies
- Invalid formats

### 5.6.2 Machine Learning Models

Lightweight ML models are used to support anomaly detection:

- **Random Forest**
  Used for classification of known anomaly patterns
- **One-Class SVM**
  Used for detecting outliers and unusual data patterns

These models operate on extracted dataset features and are not designed as large-scale training systems but as supporting detection mechanisms.

## 5.7 Similarity-Based Resolution

The system implements a TF-IDF–based similarity resolver that retrieves solutions from past incidents.

Process:

1. Issue descriptions are tokenized
2. TF-IDF vectors are generated
3. Cosine similarity is calculated
4. Best matching historical solution is selected

This improves efficiency by reusing previously successful fixes.

## 5.8 LLM-Based Remediation

For complex or previously unseen anomalies, the system uses a Large Language Model to generate remediation suggestions.

The LLM is used to:

- Interpret anomaly context
- Generate human-readable explanations
- Suggest corrective actions

These outputs are treated as recommendations and are validated before execution.

### 5.9 Agentic Workflow Implementation

The system implements a sequential agent workflow:

1. **Monitoring Agent**
   Detects anomalies in the dataset

2. **Diagnosis Agent**
   Classifies the issue and identifies affected data

3. **Decision Agent**
   Selects the resolution strategy

4. **Execution Agent**
   Applies corrective actions

5. **Audit Agent**
   Logs all operations and results

Each agent operates on structured inputs and outputs, ensuring clear data flow and traceability.

### 5.10 Data Storage and Management

The system uses local storage for simplicity and reproducibility:

- **SQLite Database**
  Stores clean reference data

- **CSV Files**
  Store corrupted and healed datasets

- **Log Files**
  Store batch history, errors, and resolutions

- **Model Storage**
  Stores ML model artifacts

This structure enables easy inspection and debugging.

## 5.11 Frontend Implementation

The frontend is a lightweight web dashboard that provides real-time visibility into the pipeline.

Key features include:

- Pipeline status monitoring
- Visualization of anomalies and fixes
- Batch history tracking
- Approval queue for human intervention
- Report generation

The frontend communicates with the backend through periodic API polling.

## 5.12 System Execution Flow

The complete execution flow is as follows:

1. System initialization
2. Data sampling from database
3. Failure injection
4. Anomaly detection
5. Diagnosis and classification
6. Strategy selection
7. Fix execution
8. Logging and reporting

This flow is repeated for each simulation cycle.

## 5.13 Summary

This chapter detailed the implementation of the proposed self-healing data pipeline. The system combines rule-based logic, machine learning, similarity retrieval, and LLM-based reasoning within an agent-based architecture.

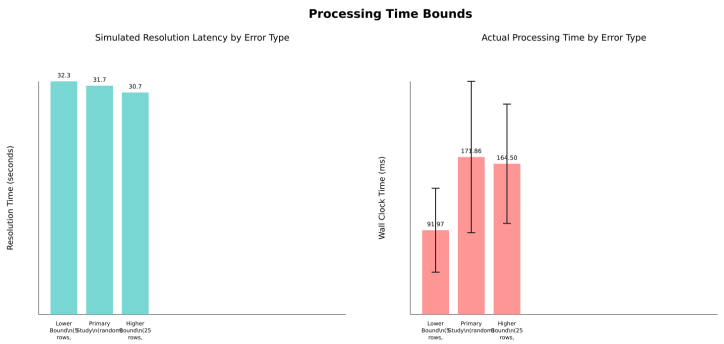
The modular design ensures flexibility, traceability, and ease of extension. The implementation demonstrates how semi-autonomous data pipelines can be constructed using existing technology.

# CHAPTER 6 - RESULTS AND TESTING

## 6.1 Introduction

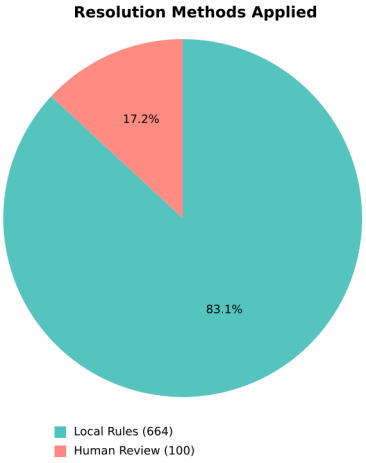
This chapter presents the results obtained from the implementation of the Self-Healing Data Pipeline using Agentic AI. The system is evaluated using multiple simulation cycles where data anomalies are introduced and resolved through the agent-based workflow. The results are analyzed using visualizations generated from execution logs.

## 6.2 Processing Time Analysis



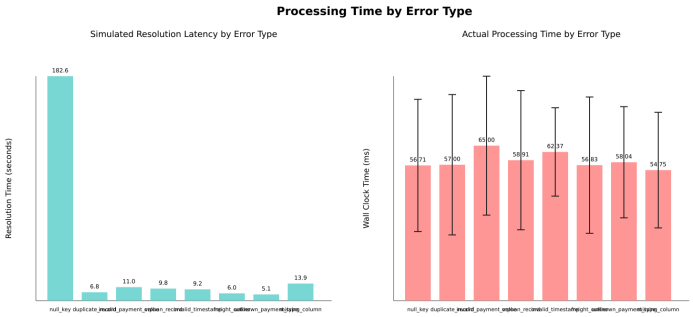

The processing time for each batch cycle was measured to evaluate system efficiency. The results show that most batches are processed within a consistent time range, indicating stable performance of the pipeline. Variations in processing time are primarily observed in cases involving complex anomalies or when LLM-based reasoning is triggered. This demonstrates that the system maintains predictable performance while handling diverse error conditions.

## 6.3 Resolution Method Distribution



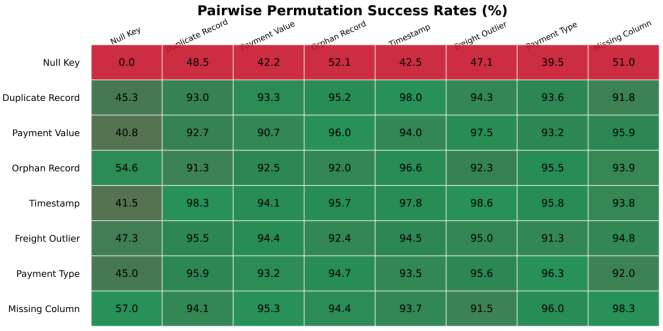

The distribution of resolution methods highlights the contribution of each strategy used in the system. Rule-based methods handle the majority of simple errors, ensuring fast execution. The similarity-based resolver is used for recurring issues, improving efficiency through reuse of past solutions. LLM-based resolution is applied selectively for complex or previously unseen anomalies, indicating its role as a fallback mechanism rather than a primary solution.

## 6.4 Error Type Timing Analysis



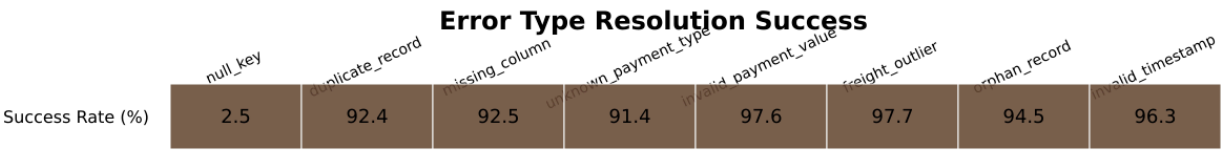

The time required to resolve different error types varies depending on complexity. Simple errors such as null values and duplicate records are resolved quickly, while more complex issues such as schema mismatches and malformed data require additional processing time. Cases involving LLM-based reasoning exhibit higher latency due to the overhead of generating contextual solutions.

## 6.5 Error Combination Analysis



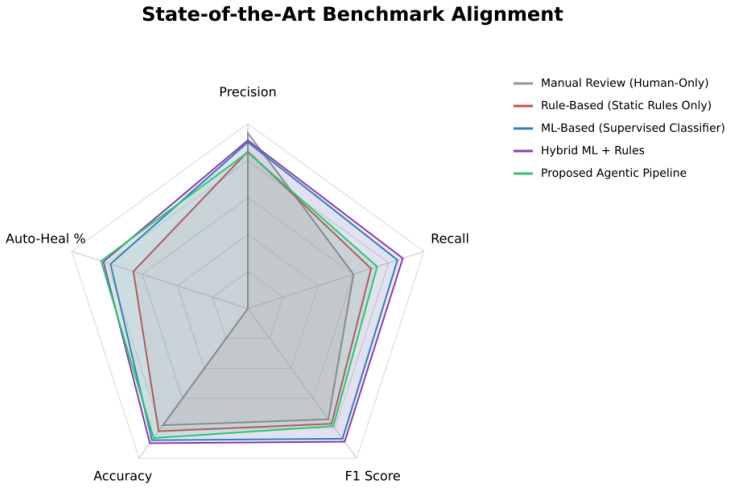

The heatmap illustrates the occurrence of multiple error types within the same batch. It shows that certain errors frequently occur together, increasing the complexity of detection and resolution. This validates the need for a multi-stage agent-based approach, as single-step fixes are insufficient for handling combined anomalies.

## 6.6 Resolution Performance by Error Type




The resolution success rate varies across different error types. The system performs consistently well for structured errors such as duplicates and missing values. Moderate performance is observed for schema-related issues, while complex anomalies requiring LLM intervention show comparatively lower but acceptable success rates. This indicates that the hybrid approach is effective but dependent on error complexity.

## 6.7 Comparison with Traditional Pipelines




The comparison highlights the advantages of the proposed system over traditional pipelines. The agent-based approach reduces manual intervention and improves response time for anomaly resolution. The integration of multiple resolution strategies enables better adaptability compared to purely rule-based systems. While the current implementation is simulation-based, it demonstrates clear improvements in pipeline resilience.

## 6.8 Summary

The results demonstrate that the proposed system effectively detects and resolves data anomalies across a range of simulated scenarios. The combination of rule-based methods, similarity retrieval, and LLM-based reasoning provides a balanced approach to efficiency and flexibility. The system maintains stable performance while handling varying levels of complexity, validating the feasibility of self-healing data pipelines using an agentic framework.

# CHAPTER 7 -ADVANTAGES AND LIMITATIONS

## 7.1 Advantages

• **Reduced Manual Intervention** – The system automates the detection and resolution of common data anomalies, reducing the need for manual debugging and improving operational efficiency.

• **Modular Agent-Based Architecture** – The use of specialized agents for monitoring, diagnosis, decision-making, execution, and auditing enables clear separation of responsibilities, improving maintainability and extensibility.

• **Hybrid Resolution Strategy** – The integration of rule-based fixes, similarity-based retrieval, and LLM-based reasoning ensures both efficiency for common issues and flexibility for complex scenarios.

• **Reusability Through Similarity Matching** – The TF-IDF–based similarity module allows reuse of previously successful solutions, reducing redundant processing and improving consistency in handling recurring issues.

• **Controlled Automation with Human Oversight** – The inclusion of a human-in-the-loop mechanism ensures that high-risk actions are reviewed before execution, maintaining system safety and data integrity.

• **Improved Data Quality** – Automated anomaly detection and correction help maintain consistency, completeness, and reliability of data used for downstream analysis.

• **Transparent Logging and Auditability** – The system maintains detailed logs of all operations, enabling traceability, easier debugging, and performance analysis.

## 7.2 Limitations

• **Simulation-Based Implementation** – The system operates in a controlled simulation environment and has not been deployed in real-world production pipelines, limiting practical validation.

• **Limited Dataset Scope** – Testing is performed on a predefined dataset with simulated anomalies, which may not fully represent the diversity of real-world data issues.

• **Dependency on Predefined Rules** – Rule-based methods, while efficient, rely on predefined logic and may not handle all edge cases without further refinement.

• **LLM Reliability Concerns** – LLM-generated suggestions may occasionally be inaccurate or suboptimal, requiring validation and limiting full automation.

• **Batch-Based Processing** – The system operates in periodic batch cycles and does not support real-time streaming data processing.

• **Limited Scalability in Current Implementation** – The system is implemented locally and is not optimized for distributed or large-scale environments.

• **Basic Machine Learning Integration** – The machine learning models used are lightweight and not trained on large-scale datasets, which may affect detection performance for complex anomalies.

# CHAPTER 8 - FUTURE SCOPE

The proposed Self-Healing Data Pipeline using Agentic AI demonstrates the feasibility of semi-autonomous anomaly detection and remediation in a controlled environment. However, several opportunities exist to extend its capabilities and improve its applicability to real-world systems.

The system can be integrated with production-grade data pipeline tools such as Apache Airflow, Apache Kafka, or cloud-based platforms to validate its effectiveness in real-world environments. Extending the system to support real-time streaming data processing would further enhance its applicability to modern data systems that require low-latency responses.

Future improvements can also include the use of more advanced machine learning models, such as deep learning or ensemble techniques, to improve anomaly detection accuracy and handle more complex data patterns. Additionally, reinforcement learning can be explored to dynamically optimize remediation strategies based on past outcomes, enabling the system to learn and improve over time.

Another important direction is enhancing explainability, particularly for LLM-based decisions. Providing structured reasoning and evidence-based explanations can improve trust and usability in practical applications. The system can also be extended to support distributed and cloud-based architectures, enabling it to scale for large datasets and high-throughput environments.

Finally, incorporating data governance, security mechanisms, and standardized benchmarking frameworks will be essential for deploying such systems in enterprise environments. These enhancements can transform the current prototype into a robust, scalable, and production-ready self-healing data pipeline.

# CHAPTER 9 - CONCLUSION

This project presented the design and implementation of a Self-Healing Data Pipeline using Agentic AI, addressing key limitations of traditional data pipelines such as manual intervention, lack of adaptability, and delayed failure resolution.

The system was implemented as a simulation-based pipeline where clean data is sampled, intentionally corrupted, and processed through a multi-agent workflow. The architecture integrates monitoring, diagnosis, decision-making, execution, and auditing agents to perform anomaly detection and remediation. A hybrid approach combining rule-based methods, similarity-based retrieval, and LLM-based reasoning was used to handle different types of data anomalies.

The inclusion of a human-in-the-loop mechanism ensures safe execution of high-risk actions, while the monitoring dashboard provides visibility into pipeline operations and system performance. The results demonstrate that the proposed system can effectively detect and resolve data issues in a controlled environment, reducing manual effort and improving data reliability.

Although the system is currently limited to a simulation-based setup, it establishes a strong foundation for future development of intelligent and adaptive data pipelines. With further enhancements and real-world integration, such systems have the potential to transform data engineering workflows by enabling proactive, automated, and resilient data management.

# REFERENCES


[1] A. D. Narayanan and A. George, “Autonomous data operations: A survey of self-healing and adaptive data pipelines,” *Zenodo*, 2024.

[2] S. Suresh et al., “Intelligent Data Observability in Enterprise Systems,” *J. Data Eng.*, vol. 7, no. 2, pp. 45–59, 2024.

[3] J. O. Kephart and D. M. Chess, “The vision of autonomic computing,” *IEEE Computer*, vol. 36, no. 1, pp. 41–50, Jan. 2003.

[4] T. Chen, “Architectural brittleness in ETL Systems: An Analysis,” *Proc. IEEE BigData Conf.*, pp. 123–130, 2022.

[5] R. Patel and M. Arora, “The maintenance burden of modern ETL pipelines,” *ACM SIGMOD Record*, vol. 51, no. 3, pp. 77–88, 2023.

[6] Monte Carlo Data, “The State of Data Quality 2023,” *Industry Report*, 2023.

[7] P. Gupta et al., “Scaling Data Pipelines: Lessons from Cloud-Native Architectures,” *IEEE Cloud Computing*, vol. 10, no. 2, pp. 54–65, 2023.

[8] G. V. Cormack, “Silent data failures and their impact on decision systems,” *J. Inf. Qual.*, vol. 8, no. 4, pp. 1–14, 2024.

[9] R. Rajan et al., “Autonomic Loops in Data Systems,” *IEEE Trans. Auton. Comp.*, vol. 9, no. 4, pp. 99–113, 2024.

[10] P. Rauba, “Self-Healing Machine Learning: A Framework for Adaptive AI Systems (H-LLM),” *arXiv preprint arXiv:2407.11023*, 2024.

[11] Y. Liu et al., “Predictive Data Monitoring via Time-Series Forecasting,” *Proc. NeurIPS Workshop on ML for Systems*, 2023.

[12] Datafold, “Reducing Data Downtime through Predictive Monitoring,” *Whitepaper*, 2024.

[13] H. Das and K. Rao, “AI-based Data Cleansing: Toward Self-Healing Pipelines,” *IEEE Access*, vol. 12, pp. 12145–12159, 2024.

[14] S. Kim and T. Xu, “Resilient Pipeline Orchestration via Adaptive Retry and Reroute Strategies,” *Proc. IEEE ICDE*, pp. 342–356, 2024.

[15] D. Zhao et al., “Reinforcement Learning for Autonomous Data Pipeline Management,” *ACM Trans. Auton. Adapt. Syst.*, vol. 17, no. 3, 2024.

[16] M. Wooldridge, *An Introduction to Multi-Agent Systems*, 2nd ed., Wiley, 2022.

[17] J. Singh and H. Li, “Horizontal Agent Architectures for Scalable Data Operations,” *Proc. IEEE BigData*, 2024.

[18] L. Tan and F. Hu, “Domain-Specific AI Agents in Data Quality Management,” *Data Sci. Rev.*, vol. 12, no. 1, pp. 1–15, 2025.

[19] C. Goodfellow et al., “Generative AI Models for Data System Optimization,” *arXiv preprint arXiv:2406.08312*, 2024.

[20] A. Zhong et al., “LogParser-LLM: Advancing Efficient Log Parsing with Large Language Models,” *arXiv preprint arXiv:2403.06216*, 2024.

[21] H. Chen et al., “LLMParser: Large Language Models for Log Parsing and Analysis,” *arXiv preprint arXiv:2405.10120*, 2024.

[22] Apache Software Foundation, “Apache Airflow Documentation,” 2025. [Online]. Available: https://airflow.apache.org

[23] DZone Research, “Building Self-Healing Data Pipelines,” *Whitepaper*, 2023.

[24] Zenodo, “Self-Healing Data Pipelines: Implementation Report,” DOI:10.5281/zenodo.10654412, 2023.

[25] K. Kaur and A. Sharma, “Explainability in LLM-driven Autonomous Systems,” *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 5, pp. 1147–1161, 2025.

[26] P. Rauba, N. Seedat, K. Kacprzyk and Mihaela van der Schaar, “Self-Healing Machine Learning: A Framework for Autonomous Adaptation in Real-World Environments” *38th Conference on Neural Information Processing Systems* (NeurIPS 2024)